

Lezione 9 - Operazioni Vettoriali

OPERAZIONI VETTORIALI

◆ Idea

Le **operazioni vettoriali** permettono di eseguire la stessa operazione su più dati contemporaneamente.

👉 invece di lavorare su un valore alla volta:

- lavoriamo su più valori insieme

✳️ eseguiamo una sola istruzione su più dati

◆ Motivazione

👉 supponiamo di avere due vettori:

- **u**
- **v**

👉 e vogliamo sommarli

👉 normalmente faremmo:

- $u_1 + v_1$
- $u_2 + v_2$
- $u_3 + v_3$
- ...

👉 quindi:

✳️ tante operazioni separate

◆ Problema

👉 questo approccio:

- richiede più istruzioni
- richiede più cicli CPU
- non sfrutta il parallelismo

◆ Idea chiave

👉 sarebbe meglio poter dire:

✳ “esegui la stessa operazione su tutte le coppie di elementi”

👉 cioè:

- una sola istruzione
- più dati

◆ Esempio concettuale

👉 invece di:

```
for (int i = 0; i < n; i++)  
{  
    C[i] = A[i] + B[i];  
}
```

👉 possiamo pensare a:

```
for (int i = 0; i < n; i += k)  
{  
    somma_k_elementi(&A[i], &B[i], &C[i]);  
}
```

👉 dove:

- k = numero di elementi elaborati insieme (es. 4, 8, 16)

✳ ogni iterazione lavora su più dati

◆ Vantaggio

👉 con operazioni vettoriali:

- leggiamo una sola istruzione
- la eseguiamo su più dati
- possiamo sfruttare il parallelismo hardware

✳ più prestazioni a parità di codice

◆ Struttura di una unità vettoriale

👉 una **unità vettoriale** funziona nel seguente modo:

- una **istruzione** viene eseguita
- più dati vengono processati in parallelo

👉 ogni dato viene elaborato da una diversa **processing unit**

◆ Funzionamento

👉 abbiamo:

- una singola istruzione
- più dati in ingresso:
 - dato 1
 - dato 2
 - dato 3
 - dato 4

👉 ogni dato viene assegnato a una diversa unità di calcolo:

- processing unit 1
- processing unit 2
- processing unit 3
- processing unit 4

👉 tutte eseguono:

✳️ la stessa operazione contemporaneamente

◆ Significato

👉 questo modello permette di:

- eseguire più operazioni in parallelo
- ridurre il numero di istruzioni
- aumentare le prestazioni

◆ Collegamento importante

👉 questo modello è:

✳️ **SIMD (Single Instruction, Multiple Data)**

📌 TASSONOMIA DI FLYNN

◆ Idea

La **tassonomia di Flynn** classifica le architetture dei calcolatori in base a:

- numero di flussi di istruzioni
- numero di flussi di dati

◆ Classi principali

👉 esistono 4 categorie:

- **SISD** (Single Instruction, Single Data)
- **SIMD** (Single Instruction, Multiple Data)
- **MIMD** (Multiple Instruction, Multiple Data)
- **MISD** (Multiple Instruction, Single Data)

◆ **SISD**

👉 modello classico:

- una sola istruzione
- un solo dato

👉 caratteristiche:

- esegue una operazione alla volta
- tipico di un singolo processore

◆ **SIMD**

👉 modello delle operazioni vettoriali:

- una sola istruzione
- più dati

👉 caratteristiche:

- stessa operazione su dati diversi
- esecuzione in parallelo

✳ base delle istruzioni vettoriali

◆ **MIMD**

👉 modello moderno:

- più istruzioni
- più dati

👉 caratteristiche:

- più unità di calcolo indipendenti
- sistemi multicore
- sistemi distribuiti

◆ MISD

👉 caso raro:

- più istruzioni
- un solo flusso di dati

👉 caratteristiche:

- usato per ridondanza
- più unità eseguono operazioni sugli stessi dati
- i risultati vengono confrontati

◆ Approfondimento su SIMD

👉 esistono diverse varianti:

- **Array processors**
 - stessa istruzione
 - memoria separata
- **Pipelined processors**
 - stessa istruzione
 - memoria condivisa
- **Associative processors**
 - esecuzione condizionale locale
 - alcune unità possono non eseguire l'istruzione

✳ nelle architetture moderne questo concetto è simile alle **istruzioni predicated**

📌 STORIA ED ESTENSIONI DELLE ISTRUZIONI VETTORIALI

◆ Storia

👉 l'idea di **calcolo vettoriale** nasce già negli anni '70

👉 uno dei primi esempi importanti:

- **Cray-1 (1975)**
 - rende popolare il concetto di **processore vettoriale**

👉 esempi precedenti:

- CDC Star-100
- TI ASC

👉 in questi sistemi:

- le operazioni lavoravano direttamente sulla memoria
- non su registri vettoriali dedicati

👉 un altro esempio importante:

- **Connection Machine CM-1 (1985)**

👉 caratteristiche:

- 65536 processori a 1 bit
- eseguono la stessa operazione su dati diversi

✳ enorme parallelismo

◆ **Evoluzione moderna**

👉 dagli anni '90:

- le architetture moderne introducono:
 - **registri vettoriali**
 - **istruzioni vettoriali**

👉 permettono di:

- lavorare su più dati con una singola istruzione

◆ Estensioni ARM

👉 ARM introduce diverse estensioni vettoriali:

- **NEON**
 - registri da 64 e 128 bit
- **SVE (Scalable Vector Extension)**
 - registri da 128 a 2048 bit
 - implementazioni reali fino a 512 bit

◆ Estensioni x86 / x86_64

👉 nelle architetture x86:

- **MMX (1997)**
 - registri da 64 bit
- **SSE (dal 1999 al 2008)**
 - registri da 128 bit
- **AVX (2011) e AVX2 (2013)**
 - registri da 256 bit
- **AVX512 (2016)**
 - registri da 512 bit
- **AVX10 (2023, proposto)**
 - unifica alcune caratteristiche di AVX512

◆ Idea importante

👉 nel tempo:

- aumenta la dimensione dei registri
- aumenta il numero di dati processati insieme

✳ sempre più parallelismo con una singola istruzione

REGISTRI VETTORIALI

◆ Idea

👉 le istruzioni vettoriali lavorano su **registri vettoriali**

👉 un registro vettoriale:

- contiene più valori
- tutti dello stesso tipo

✳️ permette di elaborare più dati con una sola istruzione

◆ Esempio

👉 consideriamo un registro da **128 bit**

👉 può contenere:

- 4 valori da 32 bit

👉 quindi:

- ogni posizione del registro contiene un valore

◆ Operazione vettoriale

👉 immaginiamo di avere:

- registro XMM0
- registro XMM1

👉 entrambi con 4 valori da 32 bit

👉 possiamo fare: **somma vettoriale**

👉 risultato:

- somma elemento per elemento

◆ Schema concettuale

👉 esempio:

$XMM0 = [a1 \ a2 \ a3 \ a4]$

$XMM1 = [b1 \ b2 \ b3 \ b4]$

👉 operazione:

$XMM2 = XMM0 + XMM1$

👉 risultato:

$XMM2 = [a1+b1 \ a2+b2 \ a3+b3 \ a4+b4]$

✳ tutte le operazioni avvengono in parallelo

◆ Struttura dei registri

👉 ogni estensione definisce:

- un insieme di registri
- un insieme di istruzioni

👉 esempi:

- SSE → registri $xmm0 - xmm15$
- AVX → registri $ymm0 - ymm15$
- AVX512 → registri $zmm0 - zmm15$

◆ Tipi di dati

👉 un registro può contenere:

- float

- interi
- altri tipi

👉 tutti gli elementi devono essere: **dello stesso tipo** di dato

◆ Problema

👉 usare direttamente assembly è scomodo

👉 quindi servono:

- strumenti più semplici
- astrazioni per usare queste istruzioni

📌 MODI PER UTILIZZARE LE ISTRUZIONI VETTORIALI

◆ Problema

👉 le istruzioni vettoriali esistono, ma:

- non sono facili da usare direttamente
- scrivere assembly è scomodo

✳️ servono metodi più pratici

◆ Opzione 1: ignorarle

👉 possiamo:

- scrivere codice normale
- lasciare che la CPU lavori in modo standard

👉 problema:

✳️ perdiamo prestazioni

◆ Opzione 2: auto-vettorizzazione

👉 i compilatori possono:

- trasformare automaticamente il codice
- usare istruzioni vettoriali

👉 esempio tipico:

```
for (int i = 0; i < n; i++)  
{  
    C[i] = A[i] + B[i];  
}
```

👉 può essere trasformato in codice vettoriale

◆ Limiti

👉 l'auto-vettorizzazione:

- non è perfetta
- basta poco per perderla
- dipende dal compilatore

✳ non sempre funziona

◆ Opzione 3: intrinsics

👉 il compilatore espone:

- funzioni speciali
- che corrispondono a istruzioni vettoriali

👉 esempio (concettuale):

```
__m256 z = _mm256_add_ps(x, y);
```

👉 vantaggio:

- controllo diretto
- *alte prestazioni*

👉 problema:

✳ non è portabile

◆ **Opzione 4: estensioni del compilatore / librerie**

👉 alcuni compilatori (es. GCC):

- offrono tipi vettoriali generici
- permettono operazioni vettoriali

👉 alcune librerie:

- nascondono i dettagli hardware

✳ *maggiore portabilità*

◆ **Opzione 5: linguaggi dedicati**

👉 esistono linguaggi progettati per SIMD

👉 esempio:

- ISPC (Intel SPMD Program Compiler)

👉 vantaggio:

- esprimono facilmente il parallelismo

◆ **Idea importante**

👉 esistono diversi modi per usare SIMD:

- automatico
- manuale (intrinsics)

- tramite librerie

✳ scelta = compromesso tra prestazioni e portabilità

INTRINSICS AVX

◆ Idea

👉 le **intrinsics** sono funzioni speciali del compilatore che rappresentano istruzioni vettoriali

👉 permettono di:

- usare SIMD direttamente in C
- evitare assembly
- avere alte prestazioni

◆ Setup

👉 per usare AVX:

```
#include <immintrin.h>
```

👉 inoltre:

- bisogna compilare con supporto AVX
- esempio: `-mavx`

◆ Tipi di dato

👉 AVX introduce tipi vettoriali

👉 esempio:

- `__m256` → contiene **8 float da 32 bit**

✳ 256 bit totali

◆ Operazioni principali

👉 alcune intrinsics importanti:

- `_mm256_loadu_ps` → *carica dati* in un registro
- `_mm256_add_ps` → *somma vettoriale*
- `_mm256_storeu_ps` → *salva dati*

◆ Esempio completo



Codice:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <immintrin.h>

float * make_array(int n)
{
    float * v = (float *)malloc(n * sizeof(float));
    for (int i = 0; i < n; i++) {
        v[i] = (float) rand() / RAND_MAX;
    }
    return v;
}

int main(int argc, char * argv[])
{
    const int n = 1<<20;
    float * a = make_array(n);
    float * b = make_array(n);
    float * c = make_array(n);

    clock_t start = clock();
    for (int i = 0; i < n; i += 8)
    {
        __m256 x = _mm256_loadu_ps(&a[i]);
        __m256 y = _mm256_loadu_ps(&b[i]);
        __m256 z = _mm256_add_ps(x, y);
        _mm256_storeu_ps(&c[i], z);
    }
    clock_t end = clock();
    float ms = (float) (end - start) / (CLOCKS_PER_SEC / 1000.0);
    printf("Somma con istruzioni vettoriali: %f ms\n", ms);

    start = clock();
    for (int i = 0; i < n; i++)
    {
        c[i] = a[i] + b[i];
    }
}
```



```
end = clock();  
ms = (float) (end - start) / (CLOCKS_PER_SEC / 1000.0);  
printf("Somma senza istruzioni vettoriali: %f ms\n", ms);  
  
return 0;  
}
```

◆ Spiegazione del codice

👉 creiamo tre array:

- a e b → input
- c → output

👉 ciclo vettoriale:

```
for (int i = 0; i < n; i += 8)
```

- lavoriamo su 8 elementi alla volta

👉 caricamento dati:

```
__m256 x = _mm256_loadu_ps(&a[i]);  
__m256 y = _mm256_loadu_ps(&b[i]);
```

- leggiamo 8 float consecutivi
- li carichiamo nei registri vettoriali

👉 operazione:

```
__m256 z = _mm256_add_ps(x, y);
```

- somma *elemento per elemento*
- eseguita *in parallelo*

👉 scrittura:

```
_mm256_storeu_ps(&c[i], z);
```

- salviamo i risultati in memoria

👉 confronto:

- primo ciclo → vettoriale
- secondo ciclo → scalare

✳ serve per confrontare le prestazioni

◆ Idea importante

👉 con AVX:

- una iterazione → 8 operazioni
- meno istruzioni
- più parallelismo

✳ miglioramento significativo delle prestazioni

📌 ESTENSIONI VETTORIALI IN GCC

◆ Idea

👉 GCC fornisce estensioni che permettono di usare tipi vettoriali direttamente in C

👉 invece di usare intrinsics:

- definiamo nuovi tipi

- il compilatore decide come implementare le operazioni

✳ *maggiore portabilità*

◆ Definizione di tipi vettoriali



Codice:

```
typedef int v4si __attribute__((vector_size (4 * sizeof(int))));  
typedef int v8si __attribute__((vector_size (8 * sizeof(int))));  
typedef int v8f __attribute__((vector_size (8 * sizeof(float))));
```

👉 significato:

- definiamo vettori di più elementi dello stesso tipo
- il compilatore li tratta come vettori

◆ Esempio completo



Codice:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef int v8si __attribute__((vector_size (8 * sizeof(int))));

int sum_vector(int * vec, int len)
{
    int sum = 0;
    for (int i = 0; i < len; i++)
    {
        sum += vec[i];
    }
    return sum;
}

int sum_vector_simd(int * vec, int len)
{
    v8si sumv = {0, 0, 0, 0, 0, 0, 0, 0};
    int i;
    for (i = 0; i+7 < len; i+= 8) {
        v8si x = *(v8si *) &vec[i];
        sumv += x;
    }
    int sum = sumv[0] + sumv[1] + sumv[2] + sumv[3] + sumv[4] + sumv[5] + sumv[6] + sumv[7];
    for (; i < len; i++)
    {
        sum += vec[i];
    }
    return sum;
}

#define N 1000000

int main(int argc, char * argv[])
{
    int v[N];
    for (int i = 0; i < N; i++)
```

```

{
    v[i] = rand() % 100;
}
clock_t start = clock();
int s = sum_vector(v, N);
clock_t end = clock();
float ms = (float) (end - start) / (CLOCKS_PER_SEC / 1000.0);
printf("s: %f\n", ms);
start = clock();
int sv = sum_vector_simd(v, N);
end = clock();
ms = (float) (end - start) / (CLOCKS_PER_SEC / 1000.0);
printf("sv: %f\n", ms);
printf("s = %d, sv = %d\n", s, sv);
return 0;
}

```

◆ Spiegazione del codice

👉 definiamo un tipo vettoriale:

```
typedef int v8si ...
```

- vettore di 8 interi

👉 funzione normale:

```
sum_vector
```

- somma elemento per elemento
- versione scalare

👉 funzione SIMD:

```
sum_vector_simd
```

👉 ciclo principale:

```
for (i = 0; i+7 < len; i+= 8)
```

- elaboriamo 8 elementi alla volta

👉 caricamento:

```
v8si x = *(v8si *) &vec[i];
```

- leggiamo 8 interi in un colpo

👉 operazione:

```
sumv += x;
```

- *somma vettoriale*

👉 riduzione finale:

```
sum = sumv[0] + ... + sumv[7];
```

- sommiamo gli elementi del vettore in modo *scalare*

👉 gestione resto:

```
for (; i < len; i++)
```

- gestiamo elementi rimanenti

◆ Confronto

👉 due approcci:

- versione scalare
- versione vettoriale

✳️ confronto delle prestazioni

◆ Idea importante

👉 queste estensioni:

- sono più semplici delle intrinsics
- più portabili
- meno controllo diretto

✳️ compromesso tra semplicità e prestazioni